

Indian Institute of Technology Dharwad

CS201L: Artificial Intelligence Laboratory

Lab 12 - Practice: Clustering (K-Means and K-Medoid)

Problem Statement

You are given with Iris flower dataset file. The file `Iris.csv` consists of 150, 4-dimensional data which includes 50 samples from each of the three species of Iris (*Iris setosa*, *Iris virginica* and *Iris versicolor*). Column 1 to 4 of the given file are the four features (attributes) that were measured from each sample: the length and the width of the sepals and petals (in centimetres) respectively. Column 5 is the class label (species name) associated with each of the samples of Iris flower. Task here is to reduce the data into 2-dimensional data using PCA and then partition (cluster) the reduced dimensional data using different clustering techniques. While performing the PCA, ignore the fifth column of the data. Use the class information in fifth column only for computing purity score.

1. Data Preprocessing and Dimensionality Reduction.

- (a) Load the Iris dataset from the CSV file.
- (b) Normalize the dataset to ensure uniform feature scaling and Apply Principal Component Analysis (PCA) and reduce the dataset to 2D using the first two principal components.
- (c) Save this 2d dataset as `iris_dataset_2D.csv` and visualize it using scatter plot.

2. K-Means Clustering.

- (a) Load the `iris_dataset_2D` datasets.
- (b) Apply K-Means clustering ($K = 3$) on the reduced iris dataset. Assign cluster labels to each data point using `KMeans.labels_`.
- (c) Visualize the clustered data using a scatter plot, coloring the points based on their assigned cluster labels. Also, show the cluster centre. (Hint: use `KMeans.cluster_centers_`).
- (d) Compute the **purity score** after examples are assigned to clusters.
- (e) Compute the **NMI** score.
- (f) Compute the **Silhouette score** to measure how similar each sample is to its own cluster compared to other clusters.
- (g) Compute the distortion measure (inertia) for number of clusters (K) as 2, 4, 6 and 10. Give the plot of K vs distortion measure. Find the heuristic optimum number of clusters using the elbow method for K-means clustering. (Hint: use `KMeans.inertia_`).

3. K-Medoid Clustering.

- (a) Load the `iris_dataset_2D` datasets.
- (b) Apply K-Medoid clustering ($K = 3$) on the reduced iris dataset. Assign cluster labels to each data point using `KMedoids.labels_`.

- (c) Visualize the clustered data using a scatter plot, coloring the points based on their assigned cluster labels. Also, show the cluster centre. (Hint: use `KMedoids.cluster_centers_`)
- (d) Compute the **purity score** after examples are assigned to clusters.
- (e) Compute the **NMI** score.
- (f) Compute the **Silhouette score** to measure how similar each sample is to its own cluster compared to other clusters.
- (g) Compute the distortion measure (inertia) for number of clusters (K) as 2, 4, 6 and 10. Give the plot of K vs distortion measure. Find the heuristic optimum number of clusters using the elbow method for K-Medoid clustering. (Hint: use `KMedoids.inertia_`)

Functions/Code Snippets:

```
# Code snippet for k mean clustering.
from sklearn.datasets import load_digits # Load the MNIST-like dataset
from sklearn.preprocessing import StandardScaler # Normalize feature
↳ scaling
from sklearn.decomposition import PCA # Dimensionality reduction using PCA
from sklearn.cluster import KMeans # Apply K-Means clustering
from sklearn_extra.cluster import KMedoids # K-Medoids clustering algorithm
from sklearn.metrics import normalized_mutual_info_score, silhouette_score #
↳ Compute NMI and Silhouette Score
import matplotlib.pyplot as plt # Visualization of clusters

# Load MNIST-like dataset
digits = load_digits()
X = digits.data # Feature matrix

# Normalize the dataset
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Reduce dimensions using PCA (for visualization)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Scatter plot of clusters
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=labels, cmap='viridis', alpha=0.6)
plt.colorbar(label="Cluster Label")
plt.title('K-Means Clustering on MNIST')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.show()

# Apply K-Means Clustering
kmeans = KMeans(n_clusters=10, random_state=42)
kmeans.fit(X_pca)
```

```
labels = kmeans.labels_  
  
# Apply K-Medoid Clustering  
kmedoids = KMedoids(n_clusters=10, random_state=42)  
kmedoids.fit(X_pca)  
labels = kmedoids.labels_  
  
# To compute distortion measure  
inertia1 = kmeans.inertia_  
inertia2 = kmedoids.inertia_  
  
# To compute NMI  
nmi = normalized_mutual_info_score(true_labels, cluster_labels)  
  
# To compute Silhouette Score  
features = X_pca  
sil_score = silhouette_score(features, cluster_labels)  
  
# To compute the purity score, follow the definition in the slide.
```

References

- Scikit-Learn K-Means documentation: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>
- Scikit-Learn K-Medoid documentation: https://scikit-learn-extra.readthedocs.io/en/stable/generated/sklearn_extra.cluster.KMedoids.html